

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO4 – Articulation (BL3)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA	Total Marks:	100
CO4 Statement:	Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.		
Student Name:	K. Pujitha	Reg. No.:	192524149
Section / Batch:	2nd year (AI & DS)	Date:	28/08/2026

Code of Conduct

I K. Pujitha (192524149) (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K. Pujitha.

Instructions

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answer using comparative analysis should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

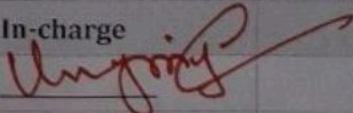
Section / Topic	Focus	Q Nos	Marks
Hashing	Linear Probing & Separate Chaining	1	20
Sorting	Merge Sort & Quick Sort	2	20
Hashing	Quadratic Probing and Double Hashing	3	20
Sorting Algorithms	Complexity Analysis	4	20
Quick Sort	Recursive and Iterative implementations	5	20
GRAND TOTAL:			100 Marks

Annexure A – Comparative Analysis

1. Compare Linear Probing and Separate Chaining for collision handling in hashing based on: (20 Marks)
 - a) Clustering effect
 - b) Memory usage
 - c) Performance under high load factor
 - d) Which method is more suitable for large-scale applications and why?
2. Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays? (20 Marks)
3. Compare Quadratic Probing and Double Hashing in terms of: (20 Marks)
 - a) Clustering issues
 - b) Collision resolution efficiency
 - c) Suitability for dense hash tables
4. Compare Best-case and Worst-case time complexity of sorting algorithms based on the following: (20 Marks)
 - a) Practical significance
 - b) Impact on algorithm selection
5. Compare Recursive and Iterative implementations of Quick Sort in terms of: (20 Marks)
 - a) Space complexity (stack usage)
 - b) Ease of implementation
 - c) Risk of stack overflow

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organization issues; lacks clarity	Poor structure and readability

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

①

Comparison of Linear Probing and Separate chaining in Hashing.

Basis	Linear Probing	Separate chaining.
① clustering effect.	Suffers from primary clustering. Consecutive occupied positions form groups, i.e. search time.	Does not suffer primary clustering because collided elements are stored in separate chains.
② Memory usage	Uses memory efficiently because all elements are stored inside the hash table itself. No extra pointers are required.	Requires extra memory for linked lists or other structures used to store collided elements.
③ Performance at high load factor.	Performance decreases significantly when the load factor becomes high. Searching & insertion require more places.	Performs relatively better at high load factor because multiple elements can be stored in the same chain.
④ Large scale applications	Suitable when the load factor is kept low and memory locality is important.	Generally more suitable for large-scale applications because it handles collisions and high load factors more effectively & flexibly.

a) clustering effect:

Linear Probing:

If a collision occurs, the algorithm checks the next available position.

$$h(k), h(k)+1, h(k)+2, \dots$$

This can create a group of consecutive occupied positions called Primary clustering.

Ex:

If several keys are mapped to positions 5, 6, 7 & 8, a cluster is formed. Future keys are more likely to encounter this cluster, increasing the number of probes.

Separate chaining:

Each hash-table position maintains a chain, usually a linked list. Colliding keys are placed in the same chain.

Ex:

Index $5 \rightarrow 25 \rightarrow 35 \rightarrow 45$.

b) Memory usage:

Linear Probing is more memory-efficient because it stores all elements directly in the hash table. It does not require additional linked list nodes or pointers.

separate chaining requires extra memory because each bucket may contain a linked list or another dynamic data structure.

Linear Probing $\rightarrow$ lower extra memory
 Separate chaining $\rightarrow$ Higher memory usage.

c) Performance under High load factor :

The load factor is :

$$\alpha = \frac{\text{Number of Elements}}{\text{Size of hash table.}}$$

In linear Probing, a high load factor causes many collisions and probes, so insertion & searching become slower.

In Separate chaining, the table can accommodate more elements in the same bucket. Performance generally degrades more gradually.

d) Which is more suitable for large-scale applications?

Separate chaining is generally more suitable for large-scale applications.

However, linear Probing can be faster in some situations when the load factor is kept low. Its elements are stored contiguously.

② Quick Sort vs Merge sort for a large Linked list

Factor	Quick sort	Merge Sort.
Time Complexity - Best	$O(n \log n)$	$O(n \log n)$
Time Complexity - Average	$O(n \log n)$	$O(n \log n)$
Time Complexity - Worst	$O(n^2)$	$O(n \log n)$
Extra memory	$O(\log n)$ average recursion stack; Can be $O(n)$ worst case.	$O(\log n)$ recursion stack, merging can be done with $O(1)$ extra node.
In-Place	Can be implemented In-Place	For linked lists, merging can be performed with copying elements.
Cache behavior	Poorer for linked lists because nodes may be scattered in memory	Better suited to sequential traversal of linked lists.
Linked-list suitability	Less suitable	Highly suitable

Why Merge sort is preferable for linked lists.
Merge sort is Generally preferred for a linked list of 10 million nodes.

For linked lists, two sorted lists can be merged simply by changing pointers:

List 1 :- $10 \rightarrow 30 \rightarrow 50$

List 2 :- $20 \rightarrow 40 \rightarrow 60$

Merged :- $10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow 50 \rightarrow 60$

Arrays:

"For arrays, the answer changes somewhat"
Quick sort is often preferred for arrays in Practical applications because:

- It can operate in-place.
- It has Good Cache locality
- It requires relatively little additional memory.
- Its average time complexity is $O(n \log n)$.

"Merge Sort" is preferable when:

- Guaranteed $O(n \log n)$ worst-case performance is required.
- Stability is important.
- Additional memory is acceptable.

3) Quadratic Probing vs Double Hashing:

Quadratic Probing and Double Hashing are open-addressing collision-resolution techniques.

Basis	Quadratic Probing	Double Hashing
a) clustering	Eliminates primary clustering but can suffer from secondary clustering.	minimizes both Primary & secondary clustering.
b) Collision resolution Efficiency	Better than linear probing but may follow predictable probe sequences	Generally more Efficient because the second hash function produces different probe sequence.
c) Dense hash tables	Performance decreases as load factor becomes high	Generally performs better in dense tables.
d) Probe sequence	$h(k) + c_1i + c_2i^2$	$h_1(k) + ih_2(k)$
Complexity	Average $O(1)$, worst $O(n)$.	Average $O(1)$, worst $O(n)$.

Clustering Issues:

Quadratic Probing:

Instead of checking consecutive positions, it uses quadratic increments.

$$h(k), h(k) + 1^2, h(k) + 2^2, h(k) + 3^2, \dots$$

Double Hashing:

It uses two hash functions:

$$h(k, i) = (h_1(k) + i \times h_2(k)) \bmod m$$

b) Collision Resolution Efficiency

Double hashing is generally more efficient because the second hash function determines the step size.

Quadratic probing has a predetermined probing pattern, whereas double hashing produces a more flexible sequence.

Double Hashing > Quadratic Probing in collision resolution efficiency.

c) Suitability for Dense Hash Tables:

As the hash table becomes dense, collisions become more frequent.

Quadratic probing may experience secondary clustering and longer probe sequences.

4. Best - Case vs worst - case Time complexity of sorting Algorithms

Practical

Best Case

Can perform

Aspect	Best Case	Worst Case
Meaning	minimum time required for favourable input	maximum time required for unfavourable input.
Practical Significance	Shows how efficiently an algorithm can perform under ideal Conditions	Shows the performance guarantee when input is difficult.
Algorithm selection	useful when input characteristics are predictable	Very important when performance guarantees are required.
Example - Quick sort	$O(n \log n)$	$O(n^2)$
Example - merge sort	$O(n \log n)$	$O(n \log n)$
Example - Insertion sort	$O(n)$	$O(n^2)$

Practical Significance:

Best Case Complexity tells us how well an algorithm can perform when the input is favourable.

For Example, Insertion sort has:

$$T_{\text{best}} = O(n)$$

worst-case Complexity tells us the maximum amount may require.

For Example, basic quick sort can have:

$$W_{\text{orst}} = O(n^2)$$

b) Impact on Algorithm Selection:

Algorithm selection depends on the nature of the input & the performance requirements.

For Example:

Quick sort: Average $O(n \log n)$, but worst-case ($O(n^2)$).

Merge Sort: $O(n \log n)$ Even in the worst case.

Insertion sort: Excellent for small or nearly sorted data but poor for large unsorted data.

Heap sort: $O(n \log n)$ worst-case and requires little extra memory.

B Recursive vs Iterative Quick Sort

Factor	Recursive Quick Sort	Iterative Quick Sort
Space Complexity	$O(\log n)$ average; $O(n)$ worst case due to recursion stack	$O(\log n)$ average with explicit stack; Can be optimized to reduce stack usage.
Stack usage	uses the system call stack automatically	uses a programmer to manage stack / data structure.
Ease of Implementation	easier & more natural	more complicated
Stack Overflow risk	Higher, Especially with Poor Pivot selection.	Low because stack size can be controlled
Code Readability	Simple & Easy to Understand	More Complex.
Control over memory	Less control	Greater control.

Q Space Complexity!

In Recursive quick sort, Every recursive call creates a stack frame.

For balanced partitions:

$$\text{Space} = O(\log n).$$

at with highly unbalanced Partitions:

$$\text{space} = O(n)$$

b) Ease of Implementation

Recursive Quick sort is Easier to Implement.
The algorithm naturally follows the divide-and-conquer structure.

QuickSort(left, right)

Partition ✓

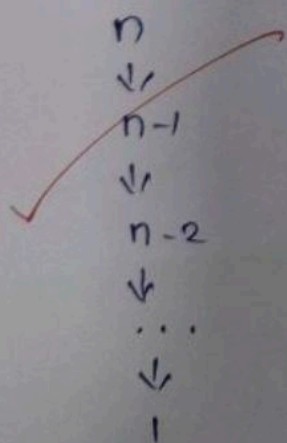
QuickSort(left Part)

QuickSort(right Part).

a) Risk of stack overflow:-

Recursive Quick sort can cause stack overflow when Partitions become highly unbalanced.

For Example



This creates approximately $O(n)$ recursive calls.

Improving →